What twenty-five years inside open source taught me about margins

I spent years on the receiving end of this pattern before I understood it.

Introduction

I have worked on open source for twenty-five years. Some of that from inside companies whose layer got commoditized, some from inside companies that benefited, a long stretch at the neutral tables where a lot of the outcomes got worked out, and today from inside Volvo Cars, where I lead the in-car software experience. It took me an embarrassingly long time to say the lesson out loud, partly because I am an engineer and I did not want it to be true.

When a large company invests heavily in open source, that investment is usually margin strategy wearing an engineering costume.

Let me put a boundary around that before anyone reaches for the reply button. Most open source is not this. Linux was a student's project before it was anyone's strategy. Python, PostgreSQL, Git, and curl were not commoditization plays, and treating them that way would be silly. The claim is about corporate capital. When a company with a balance sheet puts serious money behind a layer, there is usually a pricing logic sitting underneath the engineering one.

The costume works like this. In most companies, open source shows up as an engineering concern. It sits in the engineering budget and the visible work is technical: patches, developers, infrastructure, maintainers, code reviews, license reviews, community events, technical solutions, and pizza-fueled hackathons. If it reaches the CEO at all, it arrives as a cost line or a legal risk. Then it gets delegated downward and never enters the room where pricing and competition are actually discussed.

Underneath the costume, something else is going on. When a software layer becomes open and standard, the price of that layer as a product tends toward zero and stays there. What remains priceable moves to what sits around it: assurance, integration, certification, lifecycle support. Red Hat built a very large business on exactly that ground, and IBM paid thirty-four billion dollars for it. What becomes hard is differentiating on the layer itself.

That is a pricing event, not an engineering event. Whoever influences which layers go open, and under what governance, is helping shape where in the stack profit can exist. Less deterministically than hindsight suggests, and I will come back to that. But everyone lives with the outcome, including the people who were never in the room.

The engine underneath

The economics are older than software. Strategists call it commoditizing your complement, a framing Joel Spolsky set out in 2002 that has aged well: demand for your product rises when its complements get cheaper, so a rational company wants its complements cheap and interchangeable while keeping its own layer scarce.

What open source adds is finality. It drives the price toward zero and turns the layer into a shared standard that no single competitor can own.

So when a large company invests heavily in open source, the question I have learned to ask is which layer that investment standardizes, and which layer the company owns that benefits. Both things are usually

true at once, and neither is a criticism. Shared infrastructure gets built because someone has a reason to fund it.

I learned to ask that question the hard way. Twice, before I ever got to ask it from the other side.

Linux and the server

In the nineties, the server operating system was one of the most profitable layers in computing. Anyone remember Solaris, AIX, and HP-UX? I do, because I worked at companies that ran them. Premium prices, hardware lock-in, significant margins.

Then in December 2000, IBM committed a billion dollars to Linux. The number covered development, services, and marketing rather than pure engineering, which people sometimes forget, but as a signal it was enormous and the strategy was stated openly. The OS profit pool was small next to what surrounded it: hardware, middleware, and above all services. Funding a shared operating system reduced the differentiation available in the OS layer across the whole industry, including for competitors whose economics depended on it, while demand moved toward the layers where IBM was strongest.

Sun's position in that layer never fully recovered, and Oracle acquired the company in 2009. I am not going to pin that on Linux alone. The dot-com collapse gutted a customer base heavily concentrated in dot-coms. Commodity x86 price-performance was overtaking SPARC on its own trajectory. The 2008 financial crisis arrived immediately before the sale. Linux mattered. It was one of several things that mattered, and anyone who tells the story with a single cause is telling it too neatly.

My seat for this one was inside telecom. At Ericsson, in the early 2000s, I worked on moving core infrastructure off our proprietary TelORB platform and onto Linux. I still have the Linux Journal and Linux Magazine covers from those years, back when putting a telecom stack on a free operating system was strange enough to be a magazine story.

What I remember most is realizing, slowly, that the value we were migrating away from was not coming back. It was leaving the OS layer for good, and everyone's margins would have to be earned somewhere else in the stack. That was the first time I understood that the interesting question is not what a large investment builds. It is which layer the investor already owns.

Android and the phone

I was at Motorola, and later Palm, when the second shift arrived. This time I was standing where it landed.

In the mid-2000s the mobile operating system was a profit layer. Palm, Nokia, Microsoft, and BlackBerry all monetized the OS one way or another. Based on everything the industry knew then, treating the OS as a durable moat was a reasonable read. I certainly read it that way.

Google approached the same layer from a different position. Its profit engine was search and advertising, and the smartphone OS increasingly determined the default search experience, which made the OS layer strategically important to a business that did not sell operating systems. So Google acquired Android and released it openly. The layer the rest of us were pricing went to zero. Handset differentiation shifted toward hardware and price, and value concentrated in the layers Google operated: search, maps, the app store, services. Google never charged for Android and never needed to. Giving it away was the strategy, and the company was fairly transparent about why.

Now, the exception: Apple kept its operating system closed, integrated it tightly with hardware it controlled, and captured the substantial majority of the industry's profits. Two strategies worked. Open a layer to move value toward one you already own, or differentiate so completely on your own layer that commoditization elsewhere cannot reach you. What did not work was sitting between the two, monetizing a layer that was becoming free without owning the ground the value was moving toward.

That is where most of us were. All of these companies were full of brilliant engineers, and most of us, me included, looked at Android and saw a free mobile operating system. What we were slower to see was a change in where value would sit. That is the difficult timing of these shifts. By the time commoditization reaches your layer, the strategy that set it in motion is already years old, and you find yourself debating engineering questions while the margin question has largely been settled somewhere else.

The other side of the table: Yocto and build systems

By 2010 I had been on the receiving end of this twice. Then I joined the Linux Foundation and started seeing these dynamics from a different place.

Commercial embedded Linux was a real business at the time. MontaVista led the category. Wind River, Mentor Graphics, Enea, and Timesys all sold embedded Linux platforms, competing on proprietary build systems and toolchains. If you were building a Linux device you either rolled your own distribution, which was miserable, or you bought a commercial stack.

Around then, the embedded Linux world converged on a shared build system through the Yocto Project. It became a Linux Foundation project in 2011, with 22 founding organizations spanning silicon vendors, tooling companies, and device makers. The motivations around that table were varied and, in the healthy way of good open source projects, not identical. Chipmakers wanted a common foundation that made it easier to build Linux devices. Tooling vendors wanted to stop each maintaining a full build system alone. What they built together became the standard, and it is a genuinely good piece of shared infrastructure that the industry still runs on.

The part I find interesting is who joined. Wind River, MontaVista, and Mentor were among the founding organizations, and their businesses sat on exactly the layer the project would standardize. They joined anyway, with their eyes open, because they could see the direction of travel and understood that helping shape a shift beats watching it happen from outside. Inside the project they could influence the standard. They could also move onto ground that would stay differentiated: long-term maintenance, security, certification evidence, and in Wind River's case the hard real-time and safety-certified layers where open source could not easily follow.

Today the major embedded Linux products are built on Yocto. The build system is no longer where these companies differentiate, and by most measures the industry is better for it. The ones that read the shift early and chose their ground deliberately are still here and doing well.

I was at the Foundation through those years, close enough to how these projects come together. This time I could see more of the board: the layer being standardized, the companies choosing how and when to engage, and the different positions they held afterward.

Kubernetes and the cloud

A version of the same pattern showed up again. Google open sourced Kubernetes and contributed it to the Cloud Native Computing Foundation, an umbrella organization established under the Linux Foundation to host it and the ecosystem that grew around it. Kubernetes became the universal standard for orchestrating cloud workloads. The orchestration layer became free and interchangeable, and demand grew for the underlying compute, which is what cloud providers actually sell.

This one does not resolve as cleanly as the others, and I think that is the most useful thing about it. Google created and donated Kubernetes and remains a distant third in cloud market share. Arguably the largest beneficiary was AWS, which neither created it nor embraced it early. Meanwhile Red Hat built OpenShift into a substantial business directly on top of the standardized layer.

Commoditizing a complement moved enormous value. However, and the main lesson from this instance, it did not move that value to the party that paid for the move.

The pattern is not a mechanism

Stating four examples in a row can make something look inevitable, so I will push against my own argument for a moment.

OpenStack was a serious, well-funded effort by serious companies to standardize a layer and shift value toward hardware and services they controlled. It was explicit about the strategic logic. It did not work as intended, and several participants exited the cloud business entirely.

Nokia open-sourced Symbian defensively and it changed nothing at all.

The lesson: Funding commoditization buys you a position in the conversation, early information, and optionality. It does not buy the outcome. These shifts are contested, often contingent, and legible mostly in hindsight. Kubernetes beating its competitors was not preordained. Neither was Linux beating the BSDs. Anyone telling you this is a lever you pull is selling something.

The asymmetry

What struck me across those years was less about any one company and more about how differently companies engage. Some arrive with a clear point of view on which layers should be shared and which should stay their own, and what governance serves the whole community. Others engage mainly at the technical level and let the strategic questions resolve around them. Both kinds of participation are valuable and the projects need both. But the first group tends to influence where the ecosystem goes, and the second more often adapts to the outcome.

I should be precise about what that influence actually is, because it is easy to be either cynical about foundations or naive about them. Neutral governance means no single member controls a project, and in the projects I worked on, that held. What active participation shapes is the agenda: what gets prioritized, what interoperates with what, which problems get solved first. Control and agenda are not the same thing, and confusing them is how people end up with a bad model of how any of this works.

The pricing decisions of entire industries get made in rooms that many participants experience as technical working groups.

These are the instances I lived through personally, and they are not the whole set. Many of you reading this will have your own, from industries I have never worked in, and I would genuinely like to hear them. But set even these few side by side and the pattern is hard to unsee. Companies that filed open source under engineering gave it engineering-level attention and engineering-level seniority. Companies that recognized it as pricing strategy gave it board-level attention. Value migrated toward the second group, in servers, then phones, then embedded tooling, then the cloud.

A new shift is already forming.

Incoming: The AI layer

The contest is running now across AI models, training infrastructure, and the frameworks beneath them. The framework layer is already largely settled, and PyTorch is the clearest case: broadly shared, governed neutrally, and no longer a place where anyone differentiates. Serving and inference are standardizing quickly.

Above that, the picture is messier. Weights are increasingly available, though availability and openness are not the same thing. Training data, training code, and post-training pipelines remain closed almost everywhere, and several widely used licenses carry usage restrictions or scale thresholds. That distinction is the entire reason the Model Openness Framework exists.

It is also why I would not yet describe the model layer as commoditizing, and I want to be honest that this cuts against the tidiness of my own argument. The frontier labs are not open-weighting their strongest systems. The gap between open and closed at the frontier has been persistent rather than closing. Open models are strong in the mid-tier, and that may turn out to be a durable equilibrium rather than a waypoint on the way to something else.

The incentives are pulling in opposite directions, which is why it is unresolved. Compute and cloud providers benefit when the model layer becomes interchangeable, because demand shifts toward infrastructure they operate. Model developers have every reason to keep differentiation somewhere durable, in data, in scale, in product surface. Those two forces are pushing against each other right now.

Licensing is moving faster than the openness question. The Linux Foundation's OpenMDW, short for Open Model, Data and Weights, is a permissive licensing framework built specifically for AI model distributions, covering weights, code, documentation, and data as a single unit rather than stitching together licenses written for source code. It was developed alongside the MOF for compatibility. Licensing clarity tends to arrive before commoditization, because it lowers the cost of adopting a shared layer, so I watch this closely regardless of where the models land.

The agentic layer has the most room to run, precisely because so little is settled. Right now every serious vendor is building its own orchestration, tool-calling conventions, memory handling, and agent-to-agent protocols. That is exactly the condition that preceded standardization in embedded build systems and in container orchestration: several parties each carrying the full cost of a layer none of them monetizes directly. The Agentic AI Foundation, an umbrella foundation under the Linux Foundation, exists to give that layer neutral ground to converge on. Model providers and cloud providers both benefit from a standard agent substrate that makes their own layers easier to reach. Pushing the other way, whoever owns the dominant agent surface owns the user relationship, and that is worth defending. I do not know which force wins. Nobody does yet, which is precisely when these things get decided.

Safety, regulation, and liability sit across all of it. They will shape the pace and the shape of the shift rather than prevent it.

What to do about it

I have spent a lot of words establishing that this matters. Here is what acting on it looks like:

1. Map your stack layer by layer and mark which layers you actually monetize. Most companies cannot produce that map on request.
2. For each layer you monetize, ask who else has an incentive to see it become free and standard. If the answer is a company much larger than you, you have a strategic question rather than an engineering one.
3. Require a written position before you join or fund a foundation project. One page explaining which layers you want shared, which you intend to defend, and what outcome would make the investment worthwhile. Most participation decisions get made without this, which is how companies end up funding the commoditization of their own layer without noticing.
4. Assign open source strategy to an executive owner whose reporting line runs outside engineering because the questions are pricing questions, and pricing decisions do not get made in engineering.
5. Route foundation and consortium participation through the same review as pricing, partnerships, or acquisitions. These are capital allocation decisions with multi-year consequences. Treating them as travel budget is a mistake.

Closing

There is another shift forming that I have deliberately left out of this essay. Vehicles and physical machines are becoming software platforms, and their stacks increasingly mirror the cloud-native stack: high-performance compute, virtualization, containers, orchestration, over-the-air deployment, a cloud backend. Every layer in that stack already has a mature open source equivalent, which is what separates it from a theoretical shift.

Given where I sit, I did not go looking for this pattern in the automotive stack. I recognized it, which is different. That one needs its own essay, and it is the next thing I am writing.

The layers are different every time. The economics rhyme.

So, after twenty-five years of learning it the slow way: open source is one of the most consequential pricing instruments in modern technology. It is also a development methodology, a community building exercise, a collaboration effort, and a genuine engineering discipline, and I would never argue otherwise. But the companies that also see the pricing instrument tend to end up better positioned than the companies that only see the rest.

The next decade will be interesting to watch. Contributing the most code, chairing the most technical committees, joining foundation boards, or having the most maintainers in a project will not decide who benefits. The advantage goes to whoever understands which rooms the pricing decisions happen in, and walks into those rooms with a position, alongside their engineering resources and their willingness to collaborate.

Disclaimer

This is a personal essay about patterns I have observed. The views here are my own, drawn from personal experiences. They do not represent the opinions, positions, or strategies of my current or any of my previous employers.

The essay was inspired by a recent conversation about platform economics, the criticality of velocity and high quality code, and the value line driving buy-versus-build decisions. These discussions should be an ongoing dialogue in every company that has an OSPO.

A little self promotion

I have been writing about open source since 1998 when I first started as a contributing author to the Linux Journal. Writing helps me clarify, codify, and communicate my thinking to a broader audience. If this is the kind of content you find useful, I invite you to visit IbrahimatLinux.com for my latest reports and essays.